Harness² on Sofie models: before vs after, with full breakdown

Date: 2026-11-04 **Benchmark:** WorkBuddy-Bench, code subset (hidden tests, no LLM judge)
Executor: `sofie-code-1` (OpenCode, Docker) **Improvers tested:** `sofie-code-0`, `sofie-think-0`

1. Verdict

No reliable improvement has been demonstrated. The pipeline now works end to end and the improver produces harness edits that survive. One changed harness reached 4/4 tests in one of two runs on the tuning task, but the repeat fell back to 1/4. On held-out tasks the changed harness made no difference.

Question	Answer
Does the pipeline run on the three Sofie models?	**Yes**, end to end, after the fixes in section 7.
Do improver edits now reach the agent?	**Yes** (before: none did).
Does the improver find the real failure?	**`think-0`: yes. `code-0`: no** (section 5).
Does the changed harness raise test scores?	**Unclear.** 1 of 4 changed-harness runs on the tuning task improved; 0 of 1 on a held-out task.

2. Setup

Item	Value
Tasks	WorkBuddy-Bench code subset (Tencent; 80 real-repo tasks), `ops` domain, in stream order
Scoring	The task's hidden tests, run inside the task container. The improver never sees the score by design.
Executor	`sofie-code-1`, OpenCode in Docker, through the local proxy
Improver	`sofie-code-0` (fast) and `sofie-think-0` at `medium` effort
Judge	Not used: the code subset is graded by tests
Budgets (per day)	`code-1` 2.5, `code-0` 2.5, `think-0` 3.0

Tasks used. Task 1 `matching_absolute_paths` (tuning; improver saw its test output). Task 3 `use_correct_runtime_dir` and task 4 `add_python_dotenv_disabled` (held-out; improver never saw them).

Departure from Google's design. Google's improver sees only the agent's transcript, never test results. For tuning task 1 I also gave the improver the hidden test output. Because of this, a gain on task 1 is inflated and is not evidence of generalization.

3. Before vs after: results

Each cell is hidden tests passed.

Task	Role	Base	INV harness (`think-0`)	DOM harness (`think-0`)
1. `matching_absolute_paths`	tuning	1/4, 1/4	1/4, 1/4	**4/4**, 1/4
3. `use_correct_runtime_dir`	held-out	0/4	0/4	not run
4. `add_python_dotenv_disabled`	held-out	8/8	skipped (budget guard)	not run

Notes:

- **Base on task 1 is stable.** It scored 1/4 in all 6 of its runs across the whole project (four pipeline runs and the two evaluation repeats here).
- **DOM harness: 1 of 2 runs passed everything.** Two runs cannot separate a real partial effect from luck. The DOM skill also contains a repo-specific fact ("verified by probing"). I did not trace whether it came from the agent's transcript or from the test output the improver was shown, so its pass may partly reflect leaked information.
- **INV harness (the general one): no change** on task 1 (both runs 1/4) or on held-out task 3 (0/4).
- **Task 4's base already passes 8/8**, so there is nothing to improve there. It is a poor test task. Its INV run did not happen because `code-1` spend passed 2.2.

Agent behaviour in the evaluation runs

Run	Steps	Skill loaded
T1 base A / B	50 / 33	no / no
T1 INV A / B	47 / 92	yes / yes
T1 DOM A / B	31 / 35	yes / yes
T3 base / INV	25 / 65	no / yes
T4 base	10	no

The agent did load the new skills when they existed, so the harness changes were used. Step counts vary a lot between repeats (for example INV 47 vs 92), so I draw no conclusion from them.

4. What the improver actually produced

`think-0` (about 20 minutes per session)

- **INV: skill `results-loss-audit`.** A general habit: when a bug says results are missing, trace the pipeline for points where two distinct inputs collapse to one key, and check that every result comes back in its original form. It also adds a guardrail saying the stopping evidence is: behaviour demonstrated, the repository's own tests pass, and the diff reviewed.
- **DOM: skill `repro-first`.** Reproduce before fixing, probe look-alike inputs, and a result-contract check ("an absolute input comes back absolute"). It includes a repo-specific fact ("verified by probing").
- Both name the real failure: the tests expect the original path form back, and the base agent returned a stripped form.

`code-0` (1 to 5 minutes per session)

- With the test output in front of it, it wrote generic skills such as `absolute_path_matcher` ("make sure absolute paths are properly matched"), which restate the task title.

- The final composed harness in that run loaded the skill 0 times and still failed the same three tests (1/4).

5. Why the improvement did not appear, by component

Component	Finding
Improver `code-0`	Fast and valid, but writes restated-title skills and does not turn failing-test output into a rule.
Improver `think-0`	Finds the real failure class, but the edits did not reliably change the score.
Executor `sofie-code-1`	Loads the skills but still often repeats the same mistake; step count and outcome vary a lot between repeats.
Evaluation size	2 runs per cell cannot show a partial effect.

6. Cost and time

Item	Amount
`code-1` spend today	2.23 of 2.5 (about 0.14 per rollout in the evaluation)
`code-0` spend today	0.65 of 2.5
`think-0` spend today	0.21 of 3.0 (much lower than the 1.5 seen earlier)
`think-0` sessions	1,238 s (INV) and 1,145 s (DOM)
`code-0` sessions	50 to 308 s
Evaluation (10 planned runs)	36 minutes for 9 runs (10:20 to 10:56), run one after another

7. What was fixed along the way (all in the local repo)

Problem	Fix	Effect
Setup and runner were Vertex-only	Patched `install.py`, `workbuddy.py`, `preflight.py`, `bridge.py` for the Sofie endpoint	Pipeline reaches the models
403 on every call	Route name sent as `openai/sofie-code-1`; changed to `sofie-code-1`	Agent runs
Stalled model stream held a rollout for about 87 minutes	15-minute rollout cap (`HARNESS2_WB_TIMEOUT`)	Stalls end
`code-1` budget ran out mid-run	Spend guard at 2.3 (2.2 in the evaluation)	No corrupted rollouts
Improver skills missing `name:`/`description:` headers were silently discarded	Automatic header repair in `tree.py`, called from `recursion.py`	Edits survive
Improver acted as the solver	Mission text states its role (`context.py`)	Fewer solver-style replies
Improver could not see why the agent failed	Optional test-output feedback for tuning tasks (`HARNESS2_FEEDBACK_TASKS`)	Improver cites the real failure

8. Limits of this evidence

- One tuning task, two held-out tasks, 1 to 2 runs per cell.
- The tuning task's improver saw its test output, so its gain is inflated.
- Task 4 was ceilinged (base 8/8) and one run was skipped.
- Spending the daily `code-1` budget in one sitting left no room to repeat.

9. Next steps

1. **Repeat the DOM and INV harnesses 5 or more times each** on task 1 to see whether the 4/4 is real, after the `code-1` limit resets or is raised.
2. **Choose held-out tasks where the base partly fails** (like task 1 or 3), and drop ceilinged ones like task 4.
3. **Run the full pipeline with `think-0` as improver** (INV, DOM and AGG), now that its edits are valid, and compare the AGG choice.
4. **Report means and spread**, not single runs.